

Grokking, Reverse-Engineered

A Mechanistic Account of Delayed Generalisation on Modular Arithmetic

Punnag Choudhury · August 2026

Abstract

A one-layer transformer trained on modular addition $(a + b) \bmod p$ exhibits *grokking*: it fits the training set almost immediately but does not generalise until tens of thousands of optimisation steps later. We reproduce this phenomenon and reverse-engineer the trained network. In the discrete Fourier basis over $\mathbb{Z}_p$ the grokked token embedding is sparse — 96% of its power sits on three frequencies — and the network's answer-determining signal is a closed-form trigonometric expression, a sum of cosines $\sum_k \cos(\omega_k(a + b - c))$, recovered to $R^2 = 0.9999$ and peaking exactly at $c = a + b$. Reduced to nothing but that formula the network still solves the task at 100%. We verify the mechanism causally — the key frequencies are necessary, sufficient, and specific — and show, via progress measures computed along the trajectory, that the circuit becomes the dominant mechanism roughly 2,200 steps *before* generalisation is visible in the test loss: the "sudden" jump is the tip of a gradual, three-phase process (memorisation → circuit formation → cleanup). Grokking requires weight decay (with none, the model memorises forever) and a minimum data fraction. Finally, every independently-trained seed learns the *same* algorithm on *different* frequencies: the clock is universal, its frequencies are seed-specific. This work reproduces and consolidates the mechanistic account of Nanda et al. (2023); the analysis primitives are unit-tested on synthetic inputs and every figure is regenerated from a clean checkout.

1. Introduction

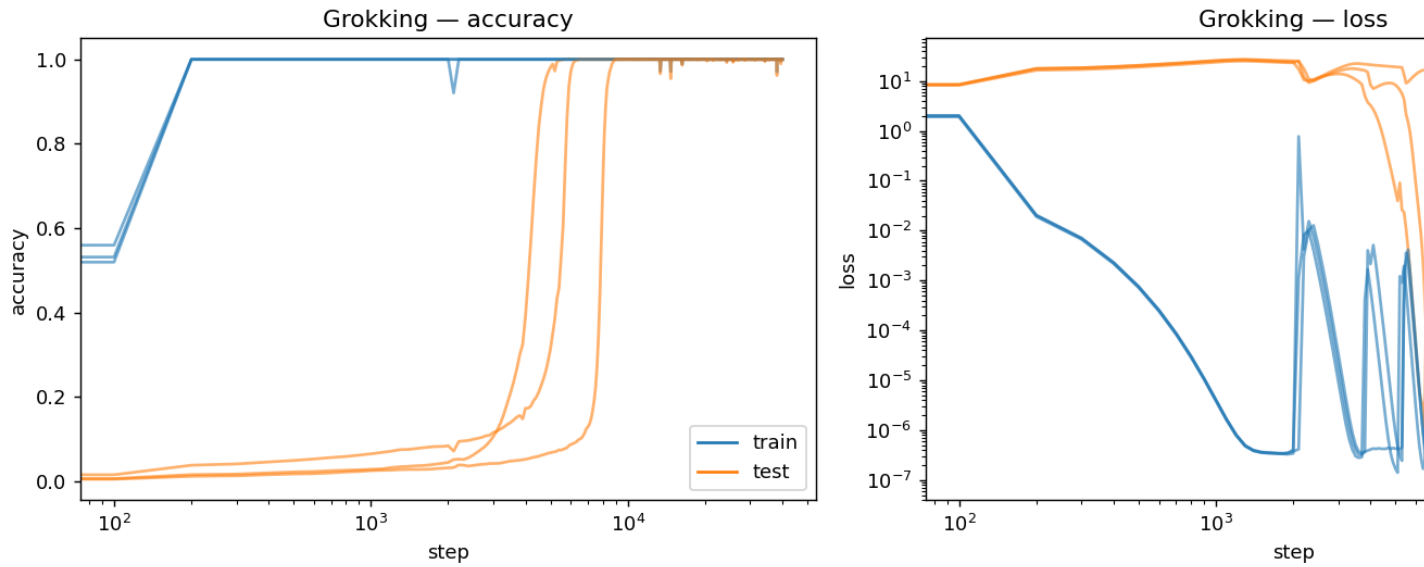
Grokking (Power et al., 2022) is a striking failure of the intuition that generalisation tracks training loss. On small algorithmic tasks a network can reach perfect training accuracy while its test accuracy remains at chance for a very long time, and then — with no change in the training signal — generalise abruptly. Because the task's ground truth is *known*, grokking is one of the few settings where "what is the network actually computing?" has a checkable answer, and where a claimed mechanism can be falsified by intervention. This report reproduces grokking on modular addition and reverse-engineers the resulting network, following the mechanistic-interpretability programme of Nanda et al. (2023).

2. Setup

The task is $(a + b) \bmod p$ for prime $p = 113$; each ordered pair (a, b) is one example, presented as the token sequence $[a, b, =]$, with the answer read off the final position. A fixed random 30% of the $p^2 = 12,769$ pairs is the training set; the rest is held out. The model is a one-layer transformer ($d_{\text{model}} = 128$, 4 heads, $d_{\text{mlp}} = 512$), written from scratch with named weight matrices and no LayerNorm so that its computation is a small, legible set of linear maps around one attention block and one MLP. It is trained full-batch with AdamW, learning rate $1e-3$, and weight decay 1.0. All analysis uses the discrete Fourier basis over $\mathbb{Z}_p$ (constant plus cos/sin at each frequency), which is orthonormal and therefore conserves power.

Grokking reproduces cleanly: across three seeds the model memorises by step 200 and generalises at steps 4,800 / 6,000 / 8,400 — a robust phenomenon with seed-dependent timing.

Phase 1 — delayed generalisation on $(a+b) \bmod 113$ (3 seed(s))

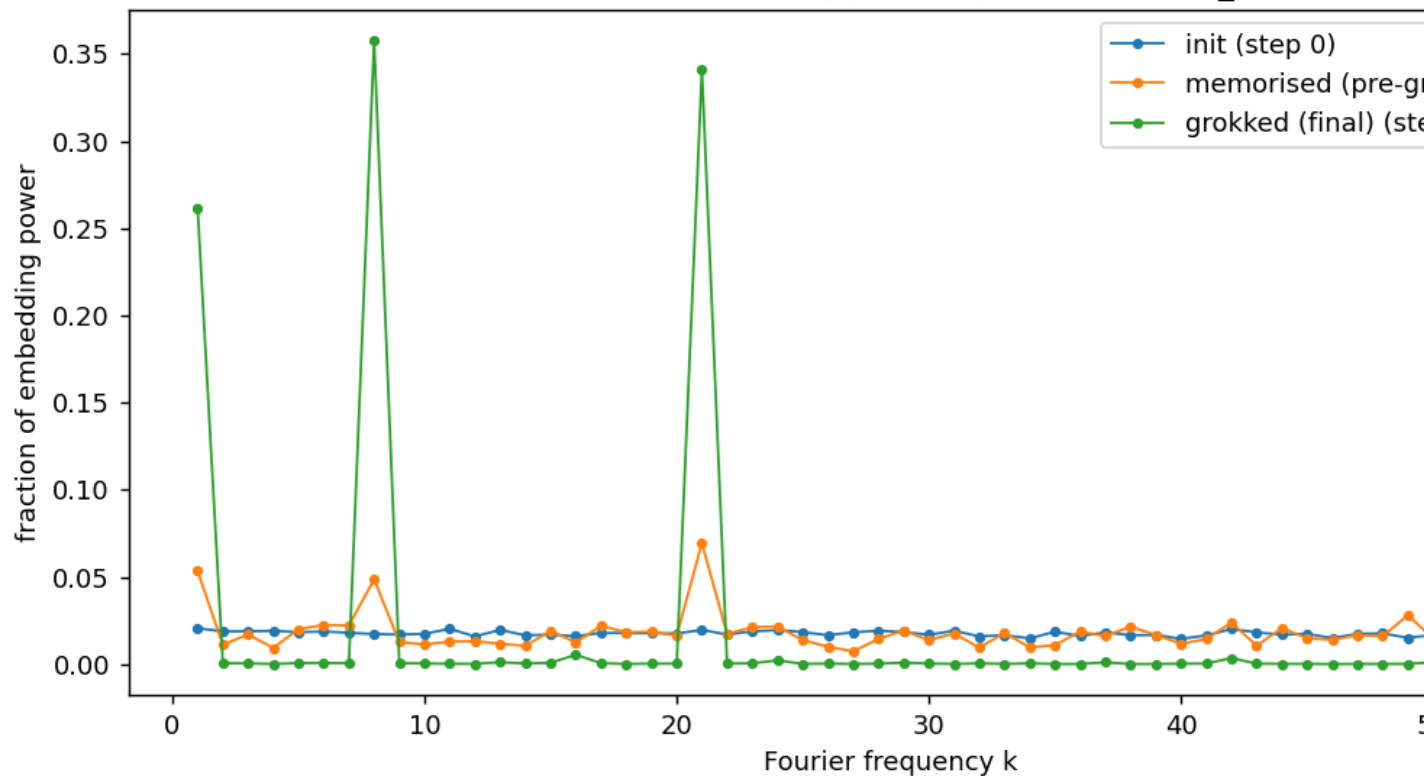


Across three seeds, train accuracy reaches 100% by step 200 while test accuracy stays at chance for thousands of steps before snapping to 100%.

3. The embedding is sparse in Fourier space

Projecting the grokked token embedding onto the Fourier basis, 96% of its power concentrates on three frequencies ($k = 1, 8, 21$), whereas at initialisation the same power is spread diffusely across all 56 frequencies. A scale-free sparsity measure, the Gini coefficient of the power spectrum, rises $0.05 \rightarrow 0.25 \rightarrow 0.93$ across initialisation, the memorisation plateau, and the grokked model. The network has learned to represent numbers using a handful of periodic features.

Phase 2 — embedding power spectrum over Z_{113}

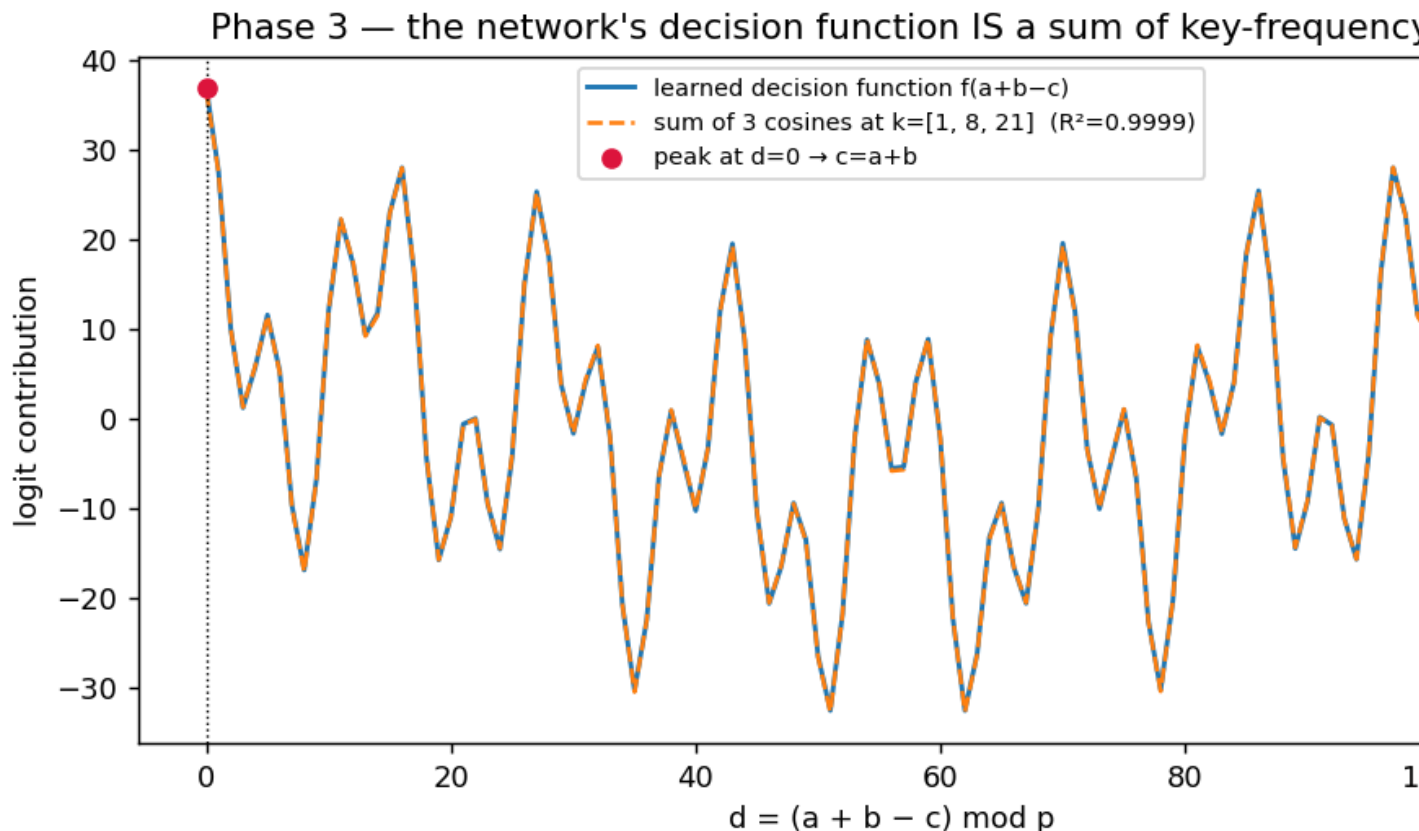


The grokked embedding (green) concentrates its power on three Fourier frequencies; the random-init (blue) and memorised-but-not-grokked (orange) embeddings are diffuse.

4. The circuit: a sum of cosines

The answer-determining signal is a pure function f of $d = (a + b - c) \bmod p$, obtained by averaging the (mean-centred) logit over all triples with a given d . Fitting $f(d)$ to a sum of cosines at the three key frequencies gives $R^2 = 0.9999$, and f is maximised at $d = 0$ — i.e. at $c = a + b$ — because every cosine interferes constructively there. Reduced to nothing but $\operatorname{argmax}_c f(a + b - c)$, this formula classifies the task at 100%.

We report the scope honestly: $f(d)$ accounts for $\sim 66\%$ of the raw centred-logit variance; the residual is c -structure not of the form $(a + b - c)$ that does not change the prediction (hence 100% accuracy despite the residual). This is the network's decision rule, expressed in closed form — the analogue, in a transformer, of reading a known formula back out of a trained model.



The learned decision function (blue) and a sum of three cosines (orange dashed) coincide at $R^2 = 0.9999$, peaking at $d = 0$ ($c = a + b$).

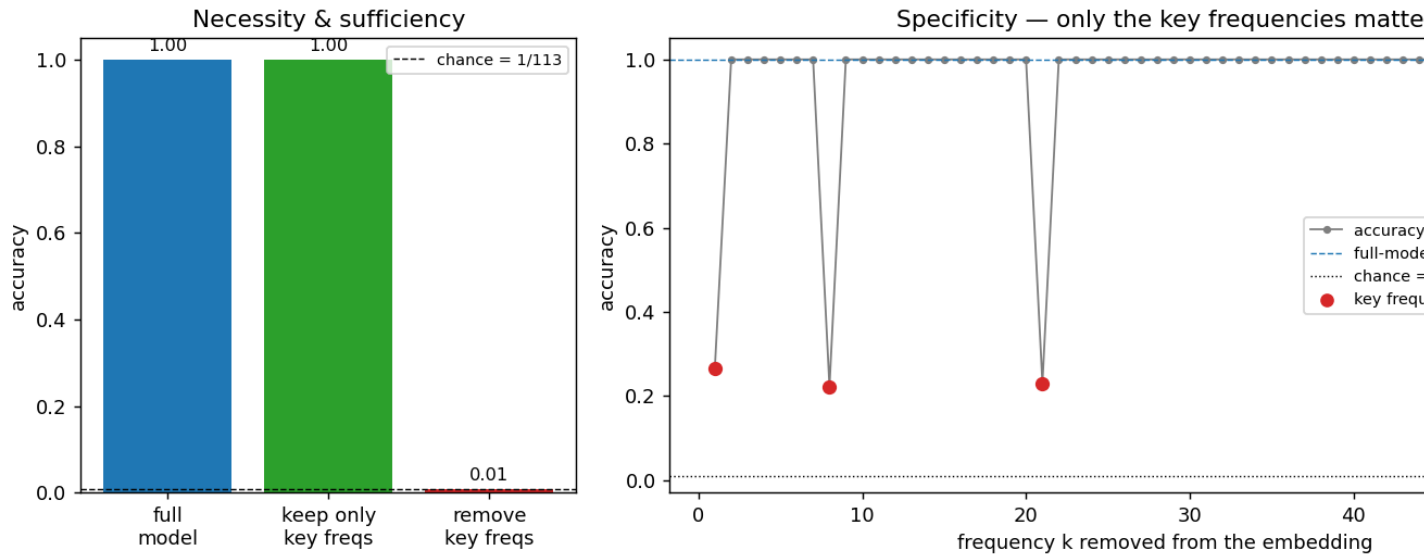
5. Causal verification

Correlation is not mechanism. We intervene directly: because the circuit reads its frequencies out of the embedding, we edit the embedding in Fourier space and re-run the otherwise-untouched network.

- **Sufficiency:** keeping *only* the three key frequencies retains 100% accuracy.
- **Necessity:** removing *only* those three collapses accuracy to chance.
- **Specificity:** removing any other single frequency has no effect.

The three frequencies the network selected are causally responsible for the entire task.

Phase 4 — the key frequencies are causally necessary and sufficient

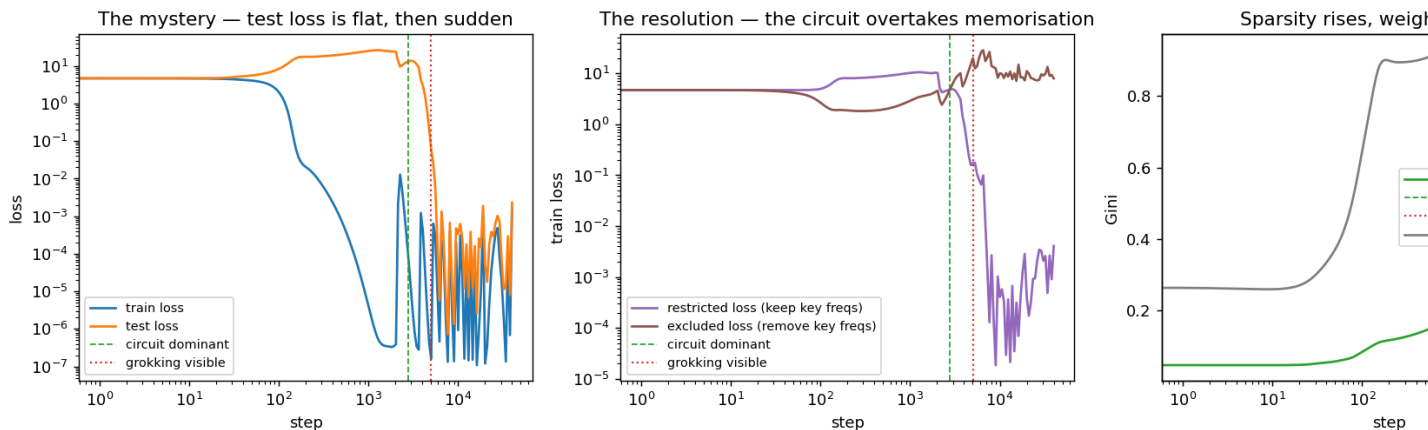


Keeping only the three key frequencies preserves 100% accuracy; removing them drops to chance; removing any other frequency does nothing.

6. Progress measures: grokking is gradual

Grokking *looks* discontinuous. Replaying the saved trajectory with the final model's key frequencies as a fixed target circuit, we compute measures that reveal it is not. The restricted loss (keep only the key frequencies) and the excluded loss (remove them) cross over at step 2,791 — the point at which the key-frequency circuit fits the data better than everything else — which is 2,222 steps before test accuracy reaches 95%. Meanwhile the embedding sparsity climbs monotonically through the plateau, and the weight norm, having grown during memorisation, decays under weight decay during the final cleanup. The three phases — memorisation, circuit formation, cleanup — are explicit, and the sudden jump is the visible tip of a continuous reorganisation.

Phase 5 — progress measures reveal grokking as a gradual, three-phase process



The sudden test-loss cliff (left) hides a gradual process: the circuit overtakes memorisation (middle, crossover) and sparsity climbs (right) well before the loss moves.

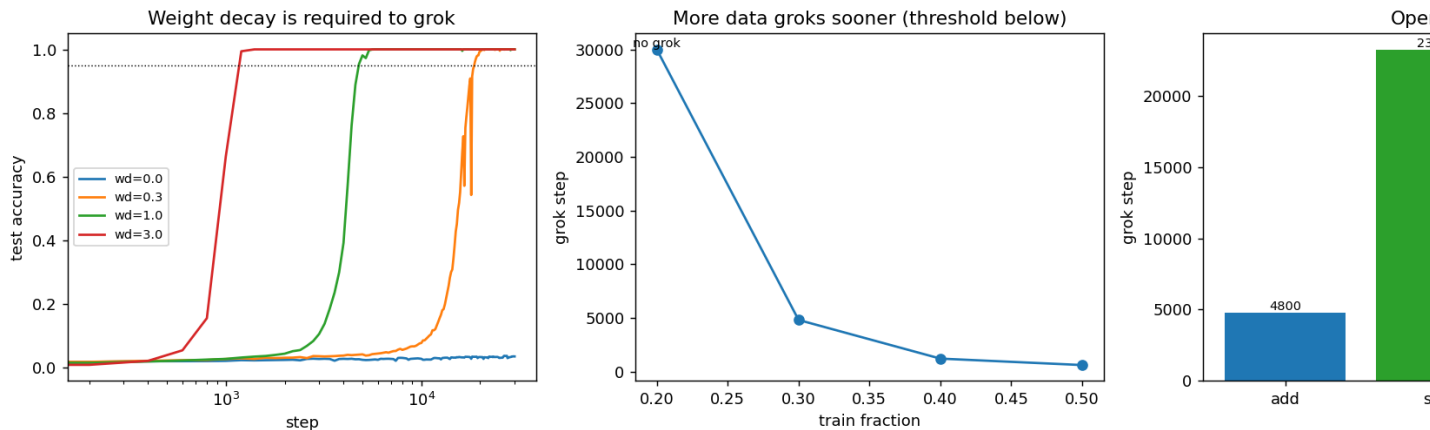
7. What grokking depends on

Varying the training recipe maps the boundaries:

- **Weight decay is essential.** With $wd = 0$ the model memorises and never groks; stronger decay groks sooner ($wd = 3/1/0.3 \rightarrow$ grok at 1,200 / 4,800 / 18,800 steps).
- **There is a data threshold.** A train fraction of 0.2 never groks within budget; 0.3 / 0.4 / 0.5 grok at 4,800 / 1,200 / 600 steps.
- **The operation matters.** Addition, subtraction and multiplication all grok to 100%, but subtraction — though group-isomorphic to addition — groks $\sim 5\times$ slower (23,200 vs

4,800 steps); isomorphic tasks need not share training dynamics.

Phase 6 — what grokking depends on: weight decay, data, and the operation

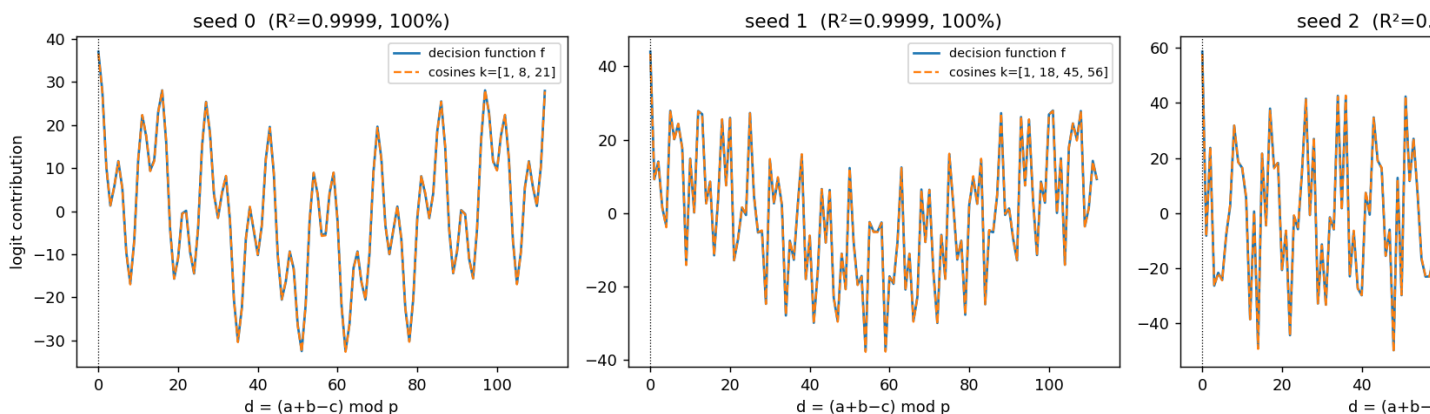


Weight decay is required to grok (left), a train-fraction threshold exists (middle), and subtraction groks far slower than addition (right).

8. Generalisation across seeds

A single reverse-engineered run could be a fluke. Running the Section 4 analysis on all three seeds, each independently learns the clock (cosine-fit $R^2 \geq 0.9998$, 100% classification, peak at $d = 0$) — but on different key frequencies (seed 0: [1, 8, 21]; seed 1: [1, 18, 45, 56]; seed 2: [13, 25, 44, 53]). The algorithm is universal; the frequencies it selects are seed-dependent. This is consistent with Zhong et al. (2023), who find that attention+MLP transformers on modular addition favour the "clock" over the "pizza" algorithm.

Phase 7 — every seed independently learns the clock (on its own key frequencies)



All three independently-trained seeds learn the same clock algorithm, each on its own set of key frequencies.

9. Limitations and further work

All experiments use a single modulus ($p = 113$); sweeping p would strengthen the universality claim and let one characterise how the number of learned frequencies scales. The 1-vs-2-layer depth ablation was not run. We do not implement a direct clock-vs-pizza discriminator (Section 8 asserts consistency rather than testing it), nor analyse the multiplication circuit, which lives on the multiplicative group and should use a log-indexed basis. This work reproduces and consolidates known results (Nanda et al., 2023; Zhong et al., 2023) rather than establishing a new mechanism; all error statements are empirical, with no formal bounds.

References

- Power, Burda, Edwards, Babuschkin & Misra (2022). *Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets*. arXiv:2201.02177.

- Nanda, Chan, Lieberum, Smith & Steinhardt (2023). *Progress Measures for Grokking via Mechanistic Interpretability*. ICLR.
- Zhong, Liu, Tegmark & Andreas (2023). *The Clock and the Pizza: Two Stories in Mechanistic Explanation of Neural Networks*. NeurIPS.

Reproducibility: every figure and number above is regenerated by the scripts in experiments/ from a clean checkout (make reproduce); the analysis primitives are unit-tested on synthetic inputs.